ITAI 1378 · COMPUTER VISION AND AI TIER 1



KeyFrame: Piano Key Press Detector

Michelle Adu-Asah

THE RECAP

Practice should feel clearer

KeyFrame gives beginners a second set of eyes between lessons.

Helping Beginner Piano Students Practice

Piano students can struggle to know whether they are hitting the right keys during solo practice.

Teachers cannot supervise every keypress constantly.

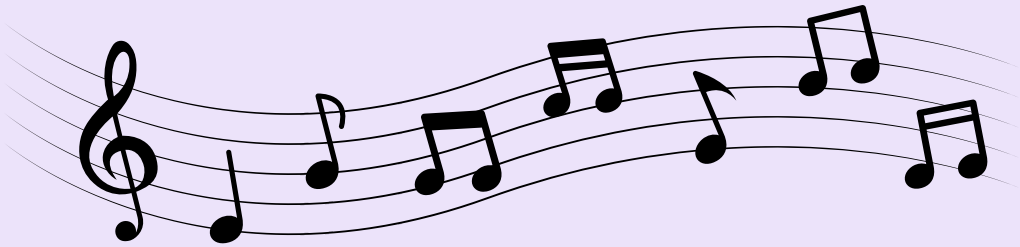
KeyFrame watches the keyboard in video and transcribes detected key hits.

Target users: beginner students, teachers, and home practice settings.

THE BUILD

From Smartphone Video to Transcribed Notes

A lightweight Python/OpenCV pipeline in Google Colab isolates piano-key regions and outputs note predictions without complex AI training



01 Video file (.mov / .mp4) enters the pipeline.

02 Frame processing accounts for camera offset.

03 Velocity filtering suppresses fast-moving hands.

04 Hold-time check tests brief touches against a minimum duration.

05 Note output: predictions such as “C4” and “D4”.

DEMO & PROOF

System in Action

Real video enters the pipeline. Candidate overlays track possible hits, fast-moving hands are suppressed, and the system generates note output.



- Demonstration of playing 2 keys G2 and C3
- 2nd video plays G1, F1 and C1

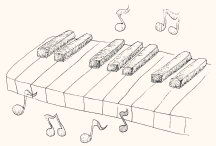


click this link to open 2nd video



UNDER THE HOOD

Three Key CV Rules for Detection



Key alignment

Dynamic offset logic keeps key regions aligned when the camera zooms or shifts.



Velocity cap

max_velocity = 8.0 filters out moving hands passing over the keys.



Hold duration

A **0.2s** minimum hold threshold helps validate real presses instead of brief touches.

DATASET

Controlled Recordings and Strict Separation

The evaluation is only meaningful when calibration material and unseen test material stay apart.

Baseline example clips

Used to calibrate hold times, velocity limits, and bounding regions.

Unseen test clips

Isolated strictly in Step 4 evaluation for genuine metric accuracy.

Reference ground truth

Checked manually, note by note, against recorded MIDI or clip labels.

MEASURING SUCCESS

Target vs. Actual Performance

PRIMARY GOAL · ACCURACY

90% target

Achieved: about 85% exact match on baseline clips.

SECONDARY GOAL · PROCESSING TIME

Under 1s

Achieved: real-time frame processing in Colab.

What helped: velocity filtering eliminated false-positive transition notes without dropping real key presses.

ADAPTATIONS

The build learned from its errors.

What Changed and Why

Added velocity filtering

Moving hands triggered extra notes, so the build added **max_velocity = 8.0** speed limits.

Dynamic key offsets

Zoomed test videos misaligned static boxes, so the build added a **key_offset** variable.

Strict loop separation

Test files leaked into early steps, so directory loops were isolated by folder prefix.

HONEST ANALYSIS

Where the System Struggles

FIXED CHALLENGE

Hand jitter double-counting

Hand movement could register the same key more than once.

Response: suppress consecutive duplicate notes.

CURRENT FAILURE CASE

Camera shift and shadow interference

Significant angle changes or heavy hand shadows can alter pixel intensity inside bounding boxes and cause missed keys.

The detector is a working proof of concept, not a replacement for a teacher.

COLLABORATION

Transparent AI Usage Log

AI supported debugging and implementation, while the build decisions stayed visible and testable.

Claude AI · Logs 1 & 2

Tested hold times, settled on **0.2s**, and debugged false predictions against ground truth.

Gemini · Logs 3 & 5

Implemented velocity filtering, isolated video loops, and fixed ground-truth swapping in Step 3.

GitHub Copilot · Log 4

Added back-to-back duplicate-note checks and key-offset options for zoomed clips.

The final system remains a transparent, testable proof of concept for beginner piano practice.

Key Lessons & Future Improvements

Lessons Learned

- Honest metric measurement on unseen test clips is critical
- Simple heuristic filters (0.2s hold duration, max velocity cap) effectively eliminate hand movement noise without complex ML

Next Steps

01

Automatic key edge detection with OpenCV contours to remove manual camera offsets or use a tripod next time

02

Lightweight CNN integration to handle variable lighting and hand shadows

Project Artifacts & Links

GitHub Repository

github.com/michelleaa68-ux/KeyFrame

Core Artifacts

- results folder: metric_table.csv, sample_predictions.txt, screenshot_example.png
- docs folder: AI_usage_log.md, presentation.pdf
- requirements.txt
- KeyFrame.ipynb notebook
- recorded video data